Introduction

Scalable language modeling is usually studied in natural language, but the same approach can also be applied to structured visual formats such as SVG. SVG files are useful for this project because they combine visual content with text-based structure. Visually, they represent icons, emojis, symbols, and glyphs. Computationally, they are XML-like sequences made of tags, attributes, coordinates, and path commands.

In this project, decoder-only Transformer models were trained to perform next-token prediction on a large SVG corpus built from StarVector datasets. After cleaning, tokenization, and length filtering, the final trainable dataset contained about 135.3M tokens. A SentencePiece BPE tokenizer with an actual vocabulary size of 2,498 was used to represent common SVG patterns while keeping sequence lengths manageable.

The project had three main goals. First, it studied whether standard Transformer scaling trends appear on SVG data by training a family of GPT-style models from tiny to xlarge under a fixed token-equivalent training budget. Second, it compared standard parameterization with μP to test whether learning-rate transfer from small to larger models improved scaling behavior. Third, it evaluated whether the best feasible model could generate syntactically valid and visually meaningful SVGs.

For the final generation experiment, the best standard model was trained for a longer run and tested in two settings: unconditional generation from an <svg prompt and prefix-conditioned generation from partial SVG drawings. Together, these experiments evaluate whether a next-token SVG model can learn both the syntax of SVG code and some of the visual structure represented by that code.

The code repository for this project is available at:
https://github.com/chanyoung-kang/svg-transformer

Data Collection and Preprocessing

The SVG language modeling dataset was built from three StarVector datasets from Hugging Face: `svg-icons-simple`, `svg-emoji-simple`, and `svg-fonts-simple`. The final cleaned corpus contained 578,997 SVG files, with 499,699 from fonts (86.3%), 75,787 from icons (13.1%), and 3,511 from emoji (0.6%). Although the project initially used the icons dataset as the core source, the final corpus was dominated by the fonts dataset, which contributed most of the token budget.

The cleaning pipeline removed XML comments, metadata, title, and description tags. It also normalized whitespace, rounded long decimal coordinates to one decimal place, filtered very

short and very long SVGs, and validated XML syntax using lxml, a Python XML parsing library. Coordinate rounding was especially important because SVG path data often contains overly precise floating point values such as 10.059374809265137. These values are usually not visually meaningful for simple icons, but they create many rare numeric patterns for the tokenizer. Rounding reduced sequence complexity while preserving the general visual structure. Render validation with CairoSVG was also considered to check whether SVGs could be converted into images, but native Cairo dependency issues in the local environment made full render validation difficult. Instead, selected SVG examples were inspected through browser based previews.

For tokenization, a SentencePiece BPE tokenizer was trained on a random sample of 200,000 cleaned SVGs. SentencePiece was chosen over ByteLevel BPE because the earlier ByteLevel tokenizer produced an overly small vocabulary and longer token sequences. In contrast, SentencePiece learned more reusable SVG and code like fragments, such as repeated tag, attribute, and path patterns. The tokenizer used a target vocabulary size of 4,096, and the actual learned vocabulary size was 2,498, including the special tokens <unk>, <bos>, <eos>, and <pad>.

After tokenization, the cleaned corpus contained 142,489,789 tokens before token length filtering. The mean SVG length was 246.1 tokens, the median was 195.0 tokens, and the maximum was 1,537 tokens. To keep training computationally manageable, SVGs longer than 1,024 tokens were removed. The final trainable dataset contained 572,701 SVG files and 135,296,594 tokens, satisfying the project requirement of at least 100M tokens.

The final dataset was split by SVG file into 98% train, 1% validation, and 1% test using a fixed random seed. This produced 132.6M training tokens, 1.37M validation tokens, and 1.32M test tokens. The split was performed after length filtering so that every SVG in the final train, validation, and test sets fit within the 1,024 token maximum sequence length.

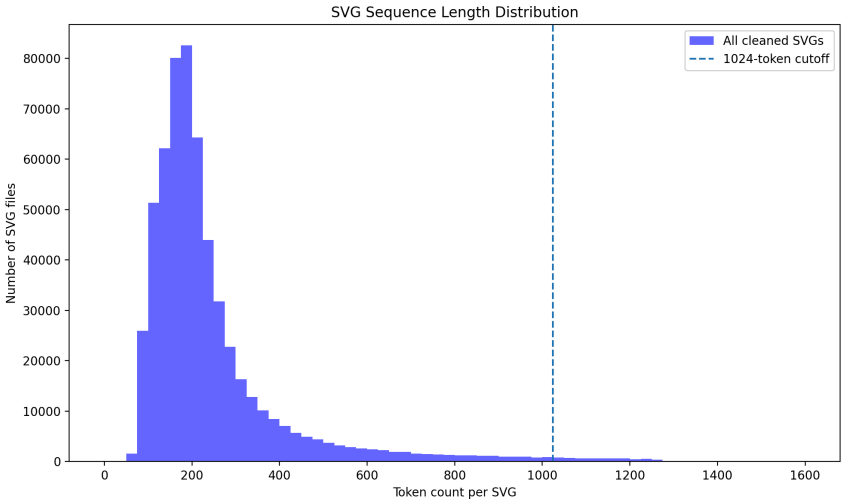

Figure 1. SVG sequence length distribution. *Most SVGs are under 400 tokens, with a long right tail. The dashed line marks the 1,024-token cutoff used for training.*

Selected SVG examples were rendered in the browser to visually inspect the cleaned dataset. These examples covered low, medium, and high complexity SVGs based on token count, where lower token SVGs tended to be simpler glyphs or icons and higher token SVGs contained more path commands and visual detail.

Figure 2. Rendered examples from the cleaned SVG corpus. The examples show the range of visual complexity in the dataset, from simple font-like glyphs to more detailed icon and emoji-style SVGs.

Methods and Experimental Setup

The SVG generation task was formulated as the next token prediction over tokenized SVG code. Given a sequence of previous SVG tokens, the model was trained to predict the next token. This setup treats SVG generation as a language modeling problem, where XML tags, attributes, numeric coordinates, and path commands are represented as tokens in a sequence. The model architecture followed a GPT style decoder only Transformer with token embeddings, positional embeddings, causal self attention blocks, feedforward layers, and a final vocabulary projection layer.

To study how model size affects SVG language modeling performance, five Transformer sizes were trained: tiny, small, medium, large, and xlarge. The model family was scaled by changing the number of Transformer layers, attention heads, embedding dimension, and total parameter count. The largest trained model was a scaled-down xlarge configuration with 34.2M parameters, rather than the suggested full 88M-parameter XL configuration. This choice was due to keeping the scaling study feasible within the available Colab GPU memory and runtime budget. Compared with the large model, the xlarge model increased depth from 8 to 10 layers while keeping the same embedding dimension and number of attention heads.

model_name	n_layer	n_head	n_embd	param_count
tiny	4	4	128	1468098
small	4	4	256	4506562
medium	6	6	384	12666818
large	8	8	512	27911618
xlarge	10	8	512	34216386

Table 1. Transformer model configurations. *The model family was scaled from tiny to xlarge by increasing the number of layers, attention heads, embedding dimension, and total parameter count.*

All models used the same tokenized SVG dataset and training setup so that differences in validation loss could be compared primarily as a function of model size. The optimizer was AdamW, and training used a batch size of 32 with a block size of 256 tokens. Each training step therefore processed 8,192 tokens. Since the encoded training split contained approximately 133.7M token IDs, one token equivalent epoch corresponded to approximately 16,324 training steps. Each model was trained for this same number of steps so that validation loss could be compared mainly as a function of model size.

Before training the full family of standard models, a learning rate sweep was performed on the smallest standard model. Several candidate peak learning rates were manually specified and tested under the same training setup. For each candidate peak learning rate, the model was trained for one token equivalent epoch, meaning it processed roughly one training set worth of tokens.

Each sweep run used the same learning rate schedule: linear warmup followed by cosine decay. During the first 5% of training steps, the learning rate gradually increased from near zero to the candidate peak value. After warmup, the learning rate decayed smoothly and ended at 10% of that peak value. This helped make early training more stable while still allowing larger updates early and smaller refinements later. After all candidate learning rates were tested, the peak learning rate with the lowest final validation loss was selected and reused for the larger standard models in the scaling study.

The first scaling experiment trained each standard model for one token equivalent epoch and compared validation loss at the end of training. Validation loss was used as the primary metric because the task is next token prediction, and validation loss directly measures how well the model predicts held out SVG tokens. The relationship between validation loss and model size was fit using the power law $L = a \times N^{(-\alpha)} + c$. Here, L is validation loss and N is the number of model parameters. The constant a controls the overall scale of the curve, α is the scaling exponent that measures how quickly loss decreases as the model gets larger, and c represents the estimated loss floor.

A second experiment compared the standard Transformer parameterization with μP, or Maximal Update Parameterization. The goal was to test whether a learning rate tuned on a small μP model would transfer more reliably to larger models. In the μP version, the model used μP specific components such as MuReadout, set_base_shapes, and MuAdam, along with adjusted attention scaling. The μP learning rate sweep was performed on the smallest μP model, and the selected μP peak learning rate was then transferred to the larger μP models without retuning. The μP models used the same tokenizer, training data, batch size, block size, model size family, and one epoch training budget as the standard models so that the comparison focused on parameterization and learning rate transfer.

For the final generation, the selected standard xlarge model was trained for 10 epochs and evaluated in two ways. First, test loss and test perplexity measured how well the model

predicted held-out SVG tokens. Test perplexity can be interpreted as the model's average number of plausible next-token choices, so lower perplexity means the model is less uncertain. Second, generated SVGs were evaluated using XML validity rate and visual inspection. XML validity checked whether each output could be parsed as well-formed XML, while visual inspection assessed whether the generated SVGs looked coherent and whether prefix-conditioned outputs completed the intended shapes.

Results and Discussion

The standard Transformer scaling experiment showed that validation loss decreased as model size increased. The tiny model had 1.47M parameters and reached a validation loss of 1.3861, while the largest trained xlarge model had 34.22M parameters and reached a validation loss of 1.1208 after one token equivalent epoch. The largest improvements occurred between the smaller models. Validation loss improved from 1.3861 for tiny to 1.2852 for small, and then to 1.1857 for medium. The improvement became smaller for the larger models, with loss decreasing from 1.1330 for large to 1.1208 for xlarge. This is expected because the trained xlarge model was only modestly larger than the large model, increasing from 27.91M to 34.22M parameters.

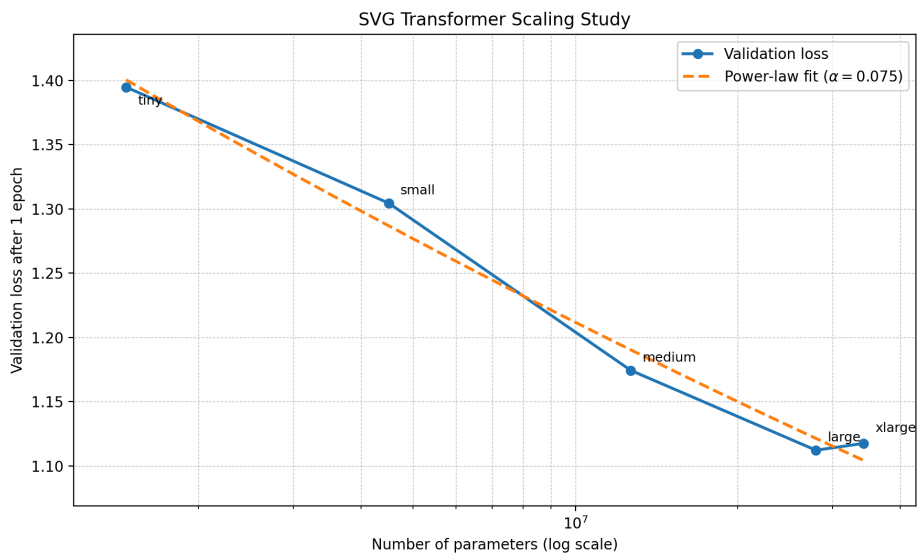

Figure 3. SVG Transformer scaling study. *Validation loss decreased as the number of model parameters increased. The fitted power law summarizes the relationship between model size and validation loss after one token equivalent epoch.*

A power law of the form L = a × N^(-α) + c was fit to the standard scaling results. The fitted scaling exponent was α = 0.075, indicating that validation loss generally decreased as model size increased, but the improvement was gradual rather than steep. The clearest gains appeared when scaling from tiny to medium. At the larger end, the trend was less clear: the xlarge model slightly underperformed the large model in this run. Since the xlarge configuration

was a scaled-down version of the suggested XL model and only modestly larger than the large model, this result should be interpreted cautiously. Overall, the standard scaling study suggests that larger models can improve SVG next-token prediction, but the high-end scaling behavior was limited by the compressed range between the large and xlarge configurations.

Training and validation curves showed similar behavior across model sizes. All models dropped sharply in loss during the early part of training, then improved more slowly over the rest of the epoch. Larger models generally reached lower training and validation losses than smaller models. The training and validation curves stayed relatively close, suggesting that there was no severe overfitting during the one epoch scaling run.

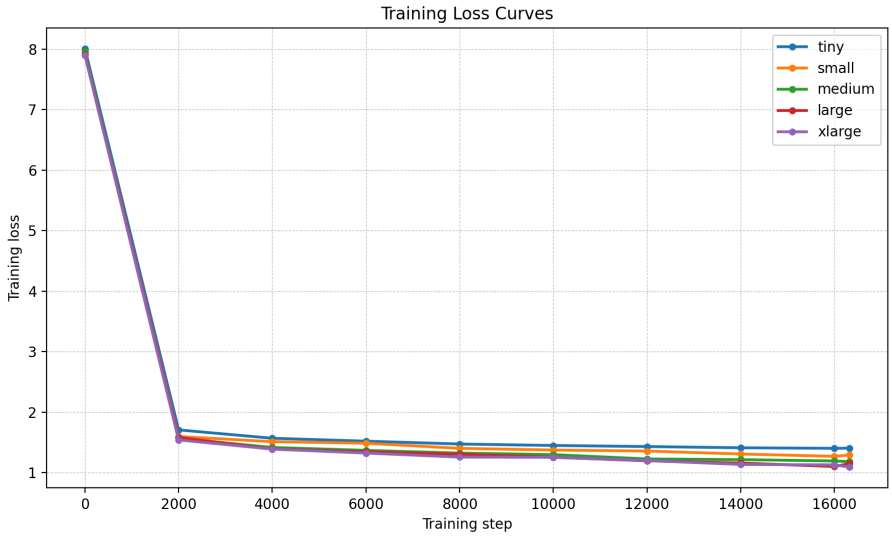

Figure 4. Training loss curves. *All models showed a sharp early decrease followed by slower improvement.*

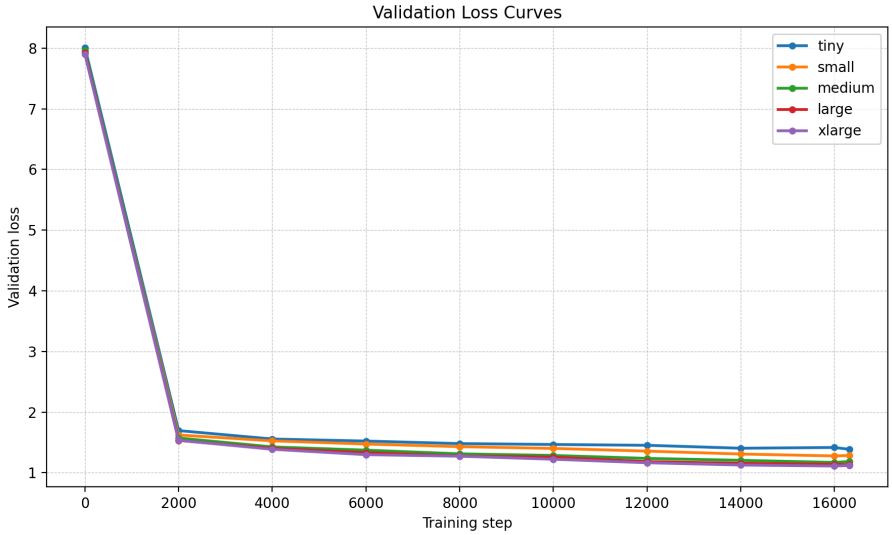

Figure 5. Validation loss curves. *Validation loss followed a similar pattern to training loss, with larger models generally reaching lower validation loss.*

The scaling runs also tracked wall clock time, throughput, and GPU memory usage. As expected, larger models required more time and memory. The tiny model trained for one epoch in 75.5 seconds, processed about 1.77M tokens per second, and used 656.6 MB of GPU memory. The xlarge model trained for one epoch in 386.7 seconds, processed about 345.8K tokens per second, and used 3,388.3 MB of GPU memory. This confirmed the tradeoff between model capacity and compute cost.

model_name	param_count	final_train_loss	final_val_loss	elapsed_sec	tokens_per_sec	gpu_memory_mb
tiny	1468098	1.4061	1.3861	75.5	1771199	656.6
small	4506562	1.2930	1.2852	90.4	1479209	856.8
medium	12666818	1.1827	1.1857	177.0	755615	1643.1
large	27911618	1.1506	1.1330	310.9	430163	2775.1
xlarge	34216386	1.0931	1.1208	386.7	345817	3388.3

Table 2. Standard Transformer training statistics. *Larger models achieved lower validation loss but required more wall-clock time and GPU memory, with lower token throughput.*

The learning rate sweep on the smallest standard model selected 8e-3 as the best peak learning rate, and this value was reused for the larger standard models. For the μP experiment, the sweep on the smallest μP model selected 2e-3. The μP implementation used μP-specific components including MuReadout, set_base_shapes, MuAdam, and adjusted attention scaling. During debugging, several implementation details were checked and revised, including the base-shape setup, optimizer parameter groups, attention scaling, and the learning-rate scheduler so that it did not overwrite MuAdam's relative learning-rate multipliers. After these fixes, the μP sweep and scaling study were rerun, but the μP models still produced much higher validation losses than the standard models.

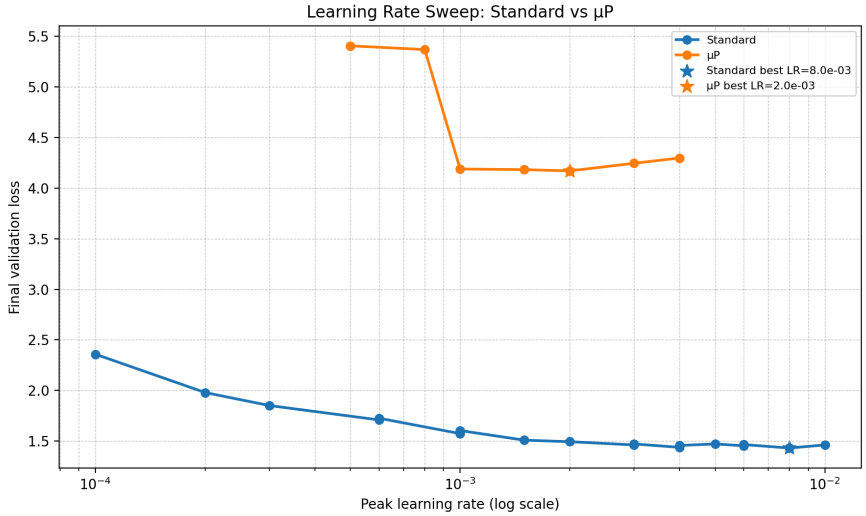

Figure 6. Learning rate sweep for standard parameterization and μP. *The standard model selected a peak learning rate of 8e-3, while the μP model selected 2e-3.*

The μP scaling results did not improve over the standard Transformer results. The standard models showed a clear decrease in validation loss as model size increased, but the μP models had much higher validation losses and did not show useful scaling improvement. For example, the standard xlarge model reached a validation loss of 1.1208, while the μP xlarge model reached 5.3806. The fitted standard scaling exponent was α = 0.124, while the μP scaling exponent was approximately 0, indicating an essentially flat scaling curve for μP. This suggests that the μP setup used here did not achieve reliable learning-rate transfer. Because μP for Transformers is sensitive to implementation details, this result should be interpreted as a limitation of this implementation rather than as a general conclusion that μP does not work.

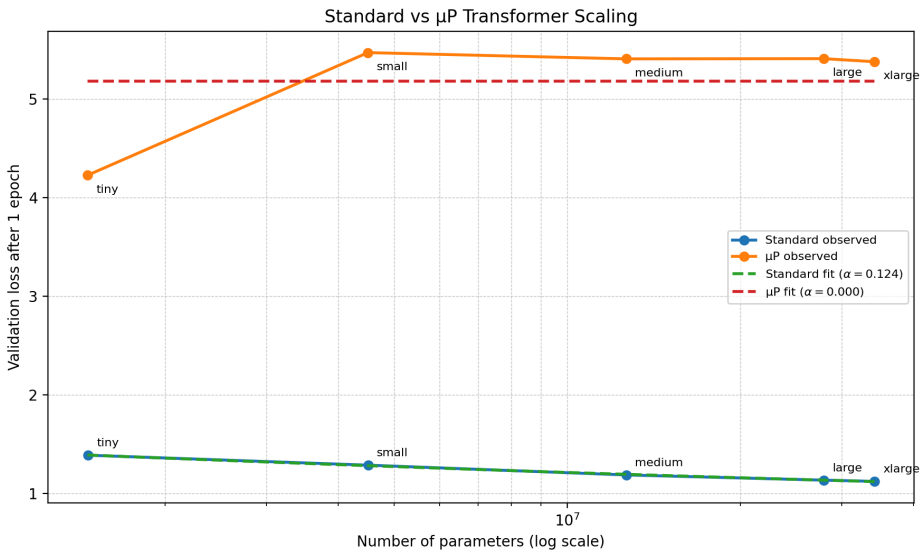

Figure 7. Standard vs. μP Transformer scaling. *The standard models showed decreasing validation loss with scale, while the μP models did not show useful scaling improvement in this implementation.*

Because the standard parameterization produced the better scaling trend, the standard scaling law was used for extrapolation. The largest trained standard model had 34,216,386 parameters, so the hypothetical 10× larger model would have 342,163,860 parameters. This 10× model was not actually trained. Instead, its expected validation loss was estimated by plugging the larger parameter count into the fitted standard power law. The predicted validation loss was 0.9784. To estimate uncertainty, bootstrap resampling was applied to the five observed standard model results, and the power law was refit multiple times. This produced an approximate 95% prediction interval of 0.9337 to 0.9830. Since the estimate is based on only five trained model sizes and extends beyond the observed range, it should be interpreted as a rough directional estimate rather than a precise forecast.

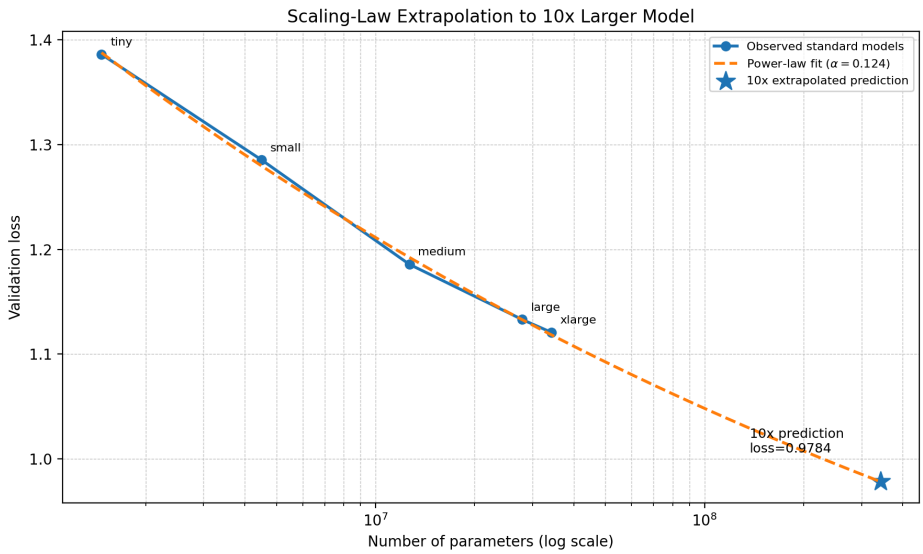

Figure 8. Scaling law extrapolation. *The fitted standard scaling law predicts a validation loss of approximately 0.9784 for a hypothetical model with 10× more parameters than the largest trained model.*

For the final generation, the standard xlarge model was trained for 10 epochs. The best checkpoint reached a validation loss of 1.0379 at step 160,000. On the held out test set, the model achieved a test loss of 1.0677 and a test perplexity of 2.9088. Since perplexity can be interpreted as the model's average number of plausible next token choices, this means the model was roughly as uncertain as choosing among about three likely next SVG tokens at each step.

The trained model was evaluated in two generation settings: unconditional generation and prefix-conditioned generation. In unconditional generation, the model was prompted only with the beginning of an SVG document, <svg, and generated the rest of the SVG. This setting tested whether the model could generate SVG-like structure mostly from scratch. Samples were generated across multiple temperatures centered around 1.0 using top-k sampling, where top-k restricts each next-token choice to the most likely candidates.

A total of 80 unconditional samples were evaluated in the final generation summary. Overall, 61 out of 80 unconditional samples were valid XML, giving an XML validity rate of 76.25%. Qualitatively, many unconditional outputs followed the basic structure of SVG code, including <svg> roots, viewBox attributes, and repeated <path> elements. However, XML validity did not always mean the rendered image was visually coherent. Some samples resembled abstract icons or glyph-like shapes, while others contained disconnected strokes or overly dense path structures.

A notable qualitative pattern was that many unconditional generations resembled letters, numbers, or font-like glyphs rather than full icon-style drawings. This is likely related to the composition of the training corpus. Although the project began with icon data, the final cleaned corpus was dominated by the fonts subset, which made up 86.3% of the SVG files. As a result,

the model appears to have learned font-like SVG structures especially strongly, which is reflected in generated samples resembling characters such as letters, numerals, and simple symbols.

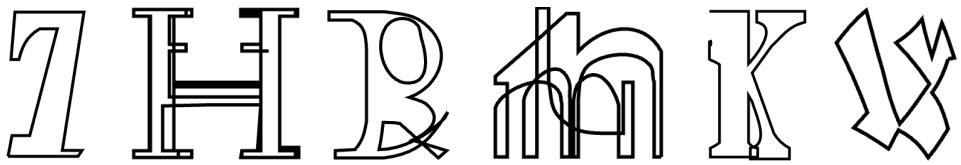

Figure 9. Unconditional SVG generation examples. *The model often generated syntactically valid SVGs from only an <svg prompt, but visual quality varied across samples. Some outputs resembled abstract icons or glyphs, while others contained disconnected or noisy path structures.*

In prefix-conditioned generation, the model was given a partial SVG prompt and asked to generate the continuation. This setting tested whether the model could use an existing visual structure as context, instead of generating from only an opening <svg prompt. The prompts included simple object-like and glyph-like drawings, such as a face missing a mouth, partial house, and partial letters. Figure 10 displays the prefix input on the left and the model-completed output on the right, making it possible to compare the intended visual prompt with the generated continuation.

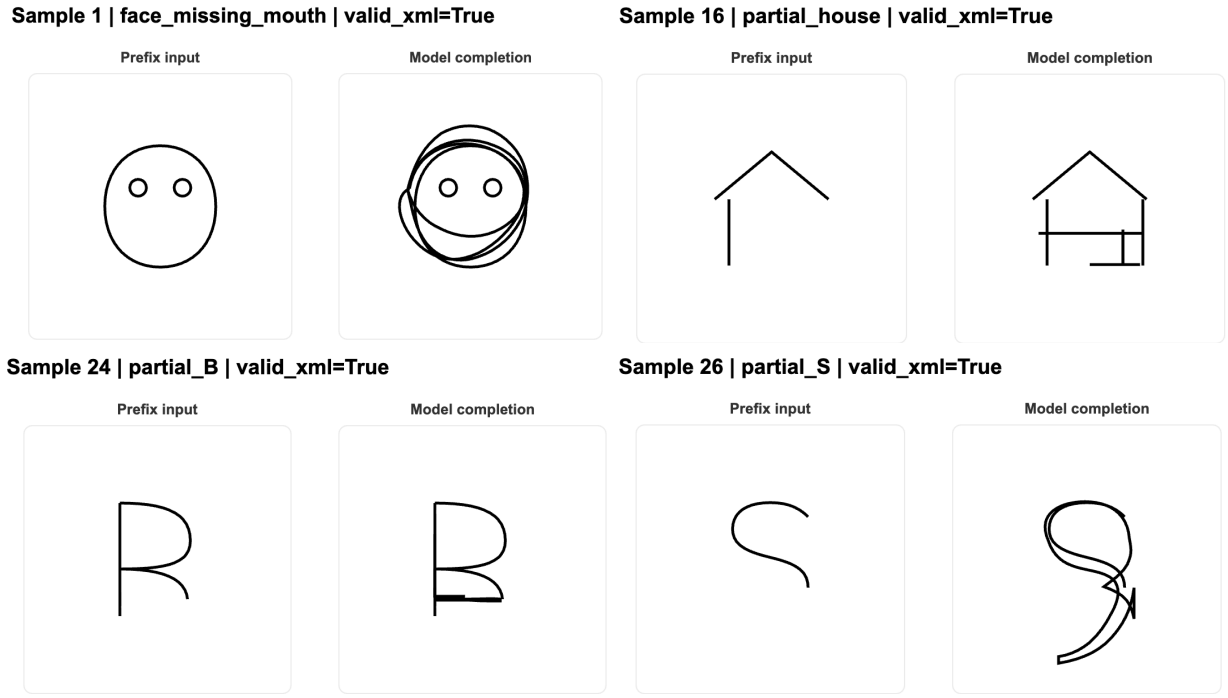

Figure 10. Prefix-conditioned SVG completion examples. *The left side shows the partial SVG prompt, and the right side shows the model completion. Some completions were valid XML and partially followed the intended structure. However, other outputs added noisy or overlapping paths, showing that the model did not reliably infer high-level visual intent from the prefix.*

Overall, prefix-conditioned generation was harder than unconditional generation and had a slightly lower XML validity rate. The model often continued valid SVG syntax and sometimes added visually related strokes, but it did not consistently complete the intended object or character. This suggests that the model was better at generating structurally valid SVG code than at understanding and completing the intended visual shape.

Conclusion

This project studied whether decoder-only Transformer language models can learn SVG generation as a next-token prediction task. The standard Transformer scaling experiment showed that validation loss generally decreased as model size increased, suggesting that scaling trends can appear in SVG language modeling. However, the gains were gradual, and the high-end trend was limited by the scaled-down xlarge configuration and available compute.

The μP experiment did not improve scaling behavior in this implementation. Although μP is intended to support learning-rate transfer across model widths, the μP models produced much higher validation losses than the standard models. This suggests that the specific μP setup used here would need further debugging or tuning before drawing broader conclusions about μP for SVG Transformers.

The final generation experiments showed that the best standard model learned meaningful SVG syntax. Many unconditional samples were valid XML and resembled glyph-like shapes, especially because the training corpus was dominated by font SVGs. Prefix-conditioned generation was more difficult: the model could often continue valid SVG code, but it did not consistently complete the intended visual structure.

The main limitations of this project were compute, dataset balance, and evaluation scope. The largest model was a scaled-down xlarge model rather than the full suggested XL configuration, so the high-end scaling trend should be interpreted cautiously. The final corpus was also dominated by font SVGs, which likely biased generations toward letters, numbers, and glyph-like shapes. Finally, generation quality was evaluated mainly through XML validity and visual inspection. Future work could train larger models for longer, use a more balanced mixture of icons, emoji, and fonts, revisit the μP implementation, and add stronger visual metrics such as SVG render rate, structural validity checks, and image-based similarity measures.